Lab 06: Time series analysis 1

Author: **N.J. de Winter** (*n.j.de.winter@vu.nl*)
Assitant Professor Vrije Universiteit Amsterdam
Statistics and Data Analysis Course

Learning goals:

- Apply and improve your knowledge of Python and Jupyter
- Learn to create and analyze time series
- Experiment with Fourier analysis and the decomposition of time series into frequencies
- Learn to apply Fourier analysis to extract dominant frequencies from time series and interpret them
- Develop a feeling for how statistical tools can help you, but you still require *your interpretation* to draw conclusions.

Introduction

In this lab, we will experiment with **time series analysis**. A **time series** is simply a dataset of which one variable is time (in any shape or form). **Time** is often the most important independent variable in the Earth Sciences, so time series analysis plays an integral role in our research.

There are many forms of data analysis that can be applied to interpret time series. In this lab, we will experience with **Fourier analysis**, or the analysis of the *frequencies* that make up a time series. *Fourier analysis* is based on the **Fourier theorem**, the mathematical discovery that any periodic time series can be decomposed into a combination of sine and cosine functions. This sounds complicated, and you do not need to know all the ins and outs of how this works. However, it can help you to have a visual understanding of how *Fourier transform* works. The video below from Youtuber *3Blue1Brown* explains the theory in a very nice visual way. You can watch it later if you want to learn some more about *Fourier analysis* so you don't lose time during the computer practical session where you can ask questions about the exercises below.

In [ ]:
```
%%HTML
<div align="center">
    <iframe width="560" height="315"
    src="https://www.youtube.com/embed/spUNpyF58BY">
    allowfullscreen
    </iframe>
</div>
```

We will work with the `numpy`, `matplotlib.pyplot` and `random` packages, as well as the modules `signal` from the `scipy` package and `interp1d` from the `scipy.interpolate` package. Make sure you have these installed in your python environment if you use Spyder. Let's start by loading the packages using the code box below. Don't forget to add the line of code needed to make the figures you create appear in `Jupyter`.

In [ ]:
```
# Make sure our figures show up in Jupyter
%matplotlib inline
# Import packages
import numpy as np
import random
import matplotlib.pyplot as plt
from scipy import signal
from scipy.interpolate import interp1d
```

Generating signals

We will start easy by creating our own time series. Create a time axis t running from 0.01 to 100 in intervals of 0.01. We will generate a periodic signal y(t), a sine wave with a period of 5 and an amplitude of 2:

In [ ]:
```
t = np.arange(0.01, 100, 0.01)
y = 2 * np.sin(2 * np.pi * t / 5)
```

Exercise 1: Use your experience with Python from the previous exercises to plot this time series in the code box below:

In [ ]:
```

```

Now, we will generate a more complex time series that is the superposition of multiple periodic components with different periods. The first signal is a sine wave with period (or wavelength) = 50 (thus frequency = 0.02), and amplitude = 2. The second sine wave has a period = 15 (frequency ~= 0.07), and amplitude = 1. The third sine wave has a period = 5, and amplitude = 0.5.

Exercise 2: Define the time axis t from 1 to 1000 in steps of 1, then create and plot the composed sine wave in the box below:

In [ ]:
```
# Create composite time series

# Plot composite time series
```

Time series in Earth science almost always contain a noise component. We will add some noise to our time series by using the `randn` function, which is part of the package `numpy.random`. We will use the `random.seed` function to set random numbers.

Exercise 3: Search the `randn` (in `numpy.random`) and `random.seed` functions in the help:

In [ ]:
```

```

The `seed` helps to remember the set of random numbers that you will create. Let's generate a time series of 1000 time steps with random noise:

In [ ]:
```
random.seed(1)
n = np.random.randn(1000)
```

Exercise 4: Now add this random noise to the y-values of the time series we created in **Exercise 2**:

In [ ]:
```
# Create yn, which adds the random noise to y2
```

Exercise 5: Now plot the original time series and the time series with random noise added together to compare the result:

In [ ]:
```

```

Time series in Earth science often have a long-term trend (e.g. related to climate change). Let's introduce a linear trend here with a time-dependent linear component:

In [ ]:
```
yt = y2 + 0.005 * t2 # Create yt, which adds a linear trend to y2
```

Exercise 6: Plot the original and time series and the time series with a trend together for comparison.

In [ ]:
```

```

Spectral analysis

Examine the `help()` for the `periodogram` function (in the `signal` package) and try to understand every input and output variable from the following line in which the `periodogram` function is applied:

In [ ]:
```
help(signal.periodogram)
```

In [ ]:
```
f0, Pxx0 = signal.periodogram(y2 ,window = np.arange(0, 1000), nfft = 1024, fs = 1)
```

Question 1: What information is contained in the newly created variable `f0`?

Answer 1:

Question 2: What information is stored in the new variable `Pxx0`?

Answer 2:

`y2` is the original time series. `0:1000` is a window for the spectral analysis, i.e. a vector with the same length as the time series. `1024` defines the number of points in the Fourier transform, which should be the next power of 2 above the signal length. `1` is the number of samples per unit of time. `Pxx` is the power spectral density (or variance) of the input signal. `f` is the frequency vector, in cycles per unit of time. `f` spans the interval between 0 and the number of samples per unit of time / 2 (here thus 0.5).

Let's plot the periodogram:

In [ ]:
```
plt.plot(f0, Pxx0)
plt.xlabel('Frequency')
```

```
plt.ylabel('Power')
plt.title('Periodogram without noise')
```

Exercise 7: Now calculate and plot the periodogram for the signal with added noise which you created in **Exercise 4**:

In [ ]:
```
# Calculate and plot the periodogram for the time series with noise.
```

Exercise 8: Let's now create a time series with five times more noise. Again, calculate and plot the periodogram.

In [ ]:
```

```

Question 3: What do you notice? Tip: For comparison, you can also try to plot both spectra in the same plot.

Answer 3:

Detrending

Let's also have a look at what the effect is of a long-term trend in a periodogram.

Exercise 9: Calculate and plot the periodogram of the time series with a trend.

In [ ]:
```

```

Question 4: What is the interpretation of the trend in the periodogram? Tip: It may help to plot the periodograms of the original time series and the time series with trend in the same plot.

Answer 4:

We can use the function `detrend` (`signal`) to get rid of a linear trend in a time series.

Exercise 10: Search for `detrend` in the `help()`, and then detrend the time series with trend and have a look at the time series before and after detrending by creating plots of both series.

In [ ]:
```

```

In [ ]:
```

```

Interpolating

Let's clear the workspace for the following exercise (only if you are using Spyder).

Exercise 11: Load the time series in the `series1.txt` and `series2.txt` files. The first column of the series contains age in kiloyears. The second column contains oxygen-isotope values measured on foraminifera. The datasets contain cyclicities of 100, 40 and 20 kyr. They look quite similar even though the numbers are slightly different. Plot both time series to explore their content.

In [ ]:
```

```

The intervals of the time vectors are not evenly spaced. Let's use the `diff` function (`numpy`, see `help()`) to calculate the intervals of the time axis for series 1. Let's also plot these intervals:

In [ ]:
```
intv1 = np.diff(series1[:,0]) # Calculate the time intervals
plt.plot(intv1) # Plot the intervals vs time
```

Let's calculate the minimum and maximum age in time series 1, and the mean interval:

In [ ]:
```
np.min(series1[:,0]) # Minimum interval
```

In [ ]:
```
np.max(series1[:,0]) # Maximum interval
```

In [ ]:
```
np.mean(intv1) # Mean interval
```

Exercise 12: Do the same for time series 2.

In [ ]:

Question 5: What do you notice when comparing these numbers for both time series?

Answer 5:

It would be nice to have both time series at equal intervals. In addition, it is always easier to work with a time series if the time interval is constant. In fact, many statistical techniques, such as *spectral analysis* will not work on time series which are not **evenly spaced**! We will try to interpolate oxygen isotope values at evenly spaced intervals to do this. We will try to keep approximately the same number of observations as the original time series (~332).

Exercise 13: Define an evenly space time vector with a time interval that approximates the mean interval of the original time series (3 kyr), and covers the range of the original data (0-997 kyr).

In [ ]:

We will use the `interp1` function from `scipy` to calculate the values of the evenly spaced time vector you created above (we call this vector `t3` for now) from the original time series. We will therefore use two different interpolation methods: `linear` and `nearest`. Let's have a look at the `interp1` function (in the `scipy` package) in the `help()` first:

In [ ]:
```
help(interp1d)
```

We will linearly interpolate series 1 as an example of how this function works:

In [ ]:
```
series1L = interp1d(series1[:,0], series1[:,1], kind='linear')(t3)
```

Exercise 14: Now use the function to interpolate series 1 and series 2, each using both the `linear` and the `nearest` method by changing the `kind` parameter.

In [ ]:

Let's now plot the original data points, and then the interpolated time series. We will also zoom in a bit (using `xlim`) so that we can clearly see the difference between both interpolation techniques:

In [ ]:
```
plt.plot(series1[:,0], series1[:,1], color = 'blue')
plt.plot(t3, series1L, color = 'red')
plt.plot(t3, series1N, color = 'green')
plt.ylabel('O isotope')
plt.xlabel('Time (kyr)')
plt.ylim((-5, 5))
plt.xlim((350, 450))
plt.legend(labels = ['Original', 'Linear interpolation', 'Nearest interpolation'])
```

Question 6: Have a good look at the code we used for plotting the interpolations above, and at the plot showing the results. Describe in your own words what the difference is between the two interpolation methods. Which one do you prefer and why?

Answer 6:

Now that we have our interpolated time series, we are ready to apply spectral analysis to study the periodicity in the oxygen isotope series.

Exercise 15: Calculate a periodogram of series 1 and series 2, and plot them. You can recycle your code from above, but think carefully about how you choose your input to the `signalperiodogram` input (especially the `window` parameter!). You can choose which interpolation method (`linear` or `nearest`) you want to use to make the data evenly spaced. Tip: You can play with the x-axis limits (`plt.xlim`) to zoom in and out in your plot to make it easier to read.

In [ ]:

Question 7: What do you observe in the periodogram plots?

Answer 7:

Question 8: Which are the dominant periodicities (in kyr) in the two data series? Explain how you calculated this result. Does the result surprise you? (Tip: Be careful when you calculate the period belonging to a peak in the powerspectrum, because your sampling interval is not equal to 1 kyr!)

Answer 8:

(Non-statistical) BONUS QUESTION: Oxygen isotope ratios in marine records are a proxy for water temperature. Can you think about a mechanism that may explain the occurrence of the periodic variability you have found in the records?

ANSWER: